Comparing Feedforward Neural Networks across Architectures and Data Dimensionalities

Jordan D. Rainford

Click Here for GitHub Code

Abstract

Feedforward Neural Networks are one of the most malleable forms of Machine Learning currently available to Data Scientists. In my exploration of this technology, I will be implementing and comparing three FFNN Models; those being Bottleneck Compression, Parallel Multi-Path Fusion, and Staged Progressive Reduction. I apply these three models to two benchmark computer vision tasks, one with a low feature count and one with a high feature count. I aim to explore how architectural decisions made in the design process of creating FFNN's can mitigate the 'curse of dimensionality' and prevent typical machine learning hurdles.

Methodology and Architecture

A Neural network is an interconnected series of neurons which send signals to each other. Smaller networks of neurons are only capable of simple pattern recognition, while larger webs of them can accomplish incredibly complex tasks.

These networks are comprised of an Input Layer, an Output Layer, and a series of 'n' Interconnected Hidden Layers. Those Hidden Layers are responsible for the entirety of the 'processing' that occurs within a Neural Network. These Hidden Layers can be organized in an infinite number of ways, but I chose to examine three distinct architectures

Table 1.

Feature / Parameter	Model 1: Bottleneck Net	Model 2: Parallel Multi-Path Net	Model 3: Staged Reduction Net
Framework	Scikit-Learn MLPClassifier	PyTorch + skorch NeuralNetClassifier	Scikit-Learn MLPClassifier
Layer Structure	256 → 128 → 64 → 128	Pre: (128 → 64) / Path A: (64 → 32) / Path B: (64 → 32 → 16 → 32) Fusion: 64 → 128 / Post: (64 → 32)	256 → 128 → 64 → 32 → 16
Structural Logic	Compression to 64, then expansion to 128	Dual parallel branches (shallow vs. deep) concatenated at fusion layer	Progressive feature contraction without expansion
Loss Function	Cross-Entropy Loss	Cross-Entropy Loss	Cross-Entropy Loss
Training Epochs	500	50	500

The Bottleneck Model

```
# Initialize Bottleneck Model (sklearn)
Bottleneck = MLPClassifier(
    hidden_layer_sizes=(256, 128, 64, 128),
    activation="relu",
    solver="adam",
    max_iter=500,
    random_state=42
)
```

The Bottleneck Model is a model which learns via compression and re-expansion of data. It begins with a large layer, one which can observe and recognize all the nuances and noise within a dataset. Then it passes to a layer half the size of the layer before, retaining slightly less nuanced information but strengthening the recognition of the most important patterns. This pattern then repeats itself for as long as the user specifies, until the analyst is left with a model which has condensed the most detailed information into only the most crucial patterns. Once the compression step is complete, the information is then passed back to a larger layer, one that can 'sharpen' the outputs and bring more clarity to compressed insights. It's a model which strips out all necessary detail and only prioritizes what is strictly necessary, only reintroducing detail with the intent of sharpening the patterns preserved by the compression.

The Parallel Model

```
# Initialize Parallel Model (skorch)
Parallel = NeuralNetClassifier(
    ParallelMultiPathNet,
    module__input_dim=X_train.shape[1],
    module__num_classes=len(np.unique(y)),
    module__pre_layers=(128, 64),
    criterion=nn.CrossEntropyLoss,
    optimizer=torch.optim.Adam,
    lr=1e-3,
    max_epochs=50,
    train_split=None,
)
```

The Parallel Model is a model which learns through multiple different channels. This model splits the 'learning' down two paths; one which prioritizes simple pattern recognition and one which prioritizes the complex. The simple, shallow path learns quickly and tosses out information that isn't strictly necessary, while the complex, deeper path only pays attention to the subtle details within the data. The idea is that when you combine the outputs of these two models, you as an analyst are left with a well-rounded, balanced model which has been trained to combat the pitfalls of both overfitting and underfitting at the same time.

The Reduction Model

```
# Initialize Custom Model (sklearn)
Reduction = MLPClassifier(
    hidden_layer_sizes=(256, 128, 64, 32, 16),
    activation="relu",
    solver="adam",
    max_iter=500,
    random_state=42
)
```

The Reduction Model is a model which learns through the compression of data. It begins with a large layer, one which can observe and recognize all the nuances and noise within a dataset. Then it passes to a layer half the size of the layer before, retaining slightly less nuanced information but strengthening the recognition of the most important patterns. This pattern then repeats itself for as long as the user specifies, until the analyst is left with a model which has condensed the most detailed information into only the most crucial patterns. Unlike the Bottleneck Model, the reduction model stops before the 'sharpening' happens. It's a model which strips out all necessary detail and only prioritizes what is strictly necessary, one which argues for simplicity above all else.

Dataset(s)

To test these models, I used two datasets on two ends of the data dimensionality spectrum. Both of which are instances of tabular dat that had been loaded from the sklearn library using Python.

Table 2.

Dataset Attribute	Scikit-Learn Digits Dataset	Olivetti Faces Dataset
Input Feature Dimensionality	64 (8x8)	4096 (64x64)
Total Samples	1,797	400
Train / Test Split	80% / 20% Stratified	80% / 20% Stratified
Training Samples	1,437	320
Testing Samples	360	80
Samples Per Class (Test)	35–37 samples	Exactly 2 samples per class

Results

Table 3.

Model Name	Test Accuracy	Macro Precision	Macro Recall	Macro F1-Score	Weighted F1-Score
Model 1: Bottleneck	**97.50%**	0.98	0.97	0.97	0.97
Model 2: Parallel Multi-Path	**96.11%**	0.96	0.96	0.96	0.96
Model 3: Staged Reduction	**97.50%**	0.98	0.97	0.97	0.97

Table 4.

Model Name	Test Accuracy	Macro Precision	Macro Recall	Macro F1-Score	Weighted F1-Score
Model 1: Bottleneck	**91.25%**	0.92	0.91	0.90	0.90
Model 2: Parallel Multi-Path	**82.50%**	0.88	0.82	0.82	0.82
Model 3: Staged Reduction	**82.50%**	0.86	0.82	0.82	0.82

Table 5.

Epoch	Digits Training Loss	Olivetti Faces Training Loss
Epoch 1	2.3073	3.6952
Epoch 10	0.3431	2.7759
Epoch 20	0.0349	0.7302
Epoch 30	0.0022	0.1417
Epoch 50	0.0004	0.0034

Insights

On High-Dimensional input spaces, the Bottleneck Model performs extremely well (8.75% lead in test accuracy). This is likely because the compression step forces the FFNN to strip away redundancies within the dataset, allowing the model itself to act as an its own feature selector. On Low-dimensional input spaces, all models performed exceptionally will in Accuracy, F1 Score, and Recall.

The Parallel Path model approached a radically low loss on the Faces dataset, while test accuracy declined by around 17%, suggesting that the model is severely overfitting to the details hidden within high-dimensionality data.

It's also worth noting that the Reduction model may be over-simplifying the model, especially for high-dimensionality data, as 16 dimensions is certainly not enough to retain enough information to confidently recognize patterns from 4000+ input features.